

Guía Práctica 4 : Nivel de Transporte

Tomás F. Melli

October 2025

Índice

1	Ejercicio 1	2
2	Ejercicio 2	2
3	Ejercicio 3	3
4	Ejercicio 4	3
5	Ejercicio 5	4
6	Ejercicio 6	5
7	Ejercicio 7	6
8	Ejercicio 8	7
9	Ejercicio 9	8
10	Ejercicio 10	10
11	Ejercicio 11	10
12	Ejercicio 12	11
13	MSL - Maximum Segment Lifetime	12

1 Ejercicio 1

¿Qué es un puerto? ¿Qué relación guarda con la multiplexación que ofrece el nivel de transporte? ¿Qué sucedería si un protocolo de nivel de transporte no implementara este concepto? ¿Qué diferencias hay entre UDP y TCP con respecto a los servicios que ofrece? ¿En qué escenarios es preferible cada uno?

Un puerto es un número lógico (0-65535) usado por el sistema operativo para distinguir múltiples conexiones o servicios que usan la misma interfaz de red. Como ocurre que dentro del mismo dispositivo físico pueden coexistir muchos programas que se comunican por red, se usan números de puerto para distinguirlos.

Cuando hablamos de multiplexación en el nivel de transporte, lo que decimos es que como la capa IP solo sabe manejar paquetes entre direcciones IP (entrega el paquete a la máquina correcta) pero no sabe a qué proceso dentro de esa máquina pertenece, a nivel de transporte, ya sea TCP o UDP, se leen los campos puerto destino y puerto origen de su encabezado y con esa información, el sistema operativo **demultiplexa**, o sea, decide a qué socket (proceso) debe entregar los datos.

El problema de no contemplar la demultiplexación es que el sistema operativo no tendría manera de distinguir qué proceso debe recibir qué dato. En particular, sería obligatorio que la comunicación sea de a una por vez.

Las diferencias entre TCP y UDP son :

Característica	UDP (User Datagram Protocol)	TCP (Transmission Control Protocol)
Orientación	No orientado a conexión.	Orientado a conexión (usa <i>handshake</i>).
Confiabilidad	No garantiza entrega ni orden.	Garantiza entrega, orden y detección de errores.
Control de flujo / congestión	No implementa.	Implementa ambos.
Encabezado	Pequeño (8 bytes).	Más grande (20 bytes mínimo).
Velocidad	Mayor (sin control).	Menor (por mecanismos de control).
Tipo de servicio	“Best effort” (sin garantía).	Confiable y con confirmaciones (ACK).

Escenario	Protocolo preferido	Justificación
Streaming (video, audio)	UDP	Tolera pérdidas; prioriza baja latencia.
DNS, VoIP, juegos online	UDP	Tramas pequeñas, rápidas, sin necesidad de confirmaciones.
Web (HTTP/HTTPS)	TCP	Se necesita confiabilidad e integridad.
Correo, transferencia de archivos (FTP, SSH, SMTP)	TCP	Es esencial que los datos lleguen completos y ordenados.

2 Ejercicio 2

Tanto a nivel de enlace como de transporte, se plantea la necesidad de garantizar comunicaciones confiables: que los datos recibidos representen exactamente toda la información transmitida y en el mismo orden. Para esto, en ambos niveles, se usan estrategias que son similares. Sin embargo, en la capa de transporte ciertos parámetros deben ir cambiando dinámicamente mientras que en la capa de enlace no.

Discuta las estrategias que usan tanto las conexiones de nivel de enlace como las de nivel de transporte

Estrategia	Nivel de enlace	Nivel de transporte (TCP)
Detección de errores	Códigos de verificación como CRC o FCS (Frame Check Sequence).	Suma de verificación (<i>checksum</i>) sobre los datos del segmento.
Retransmisión	Si una trama se pierde o detecta error, se retransmite (ARQ: <i>Automatic Repeat reQuest</i>).	Retransmite segmentos no confirmados (usa ACK y temporizadores).
Numeración de tramas/segmentos	Se numeran tramas para detectar duplicados o pérdidas.	Números de secuencia en bytes para reconstruir el flujo y detectar pérdidas.
Confirmaciones (ACKs)	El receptor envía ACKs para confirmar la recepción de tramas correctas.	TCP usa ACK acumulativos para confirmar todos los bytes recibidos.
Control de flujo	<i>Stop-and-wait</i> o ventana deslizante (tamaño fijo).	Ventana deslizante con tamaño dinámico (control de flujo y congestión).

Discuta las diferencias entre ambos niveles. ¿A qué se deben?

	Nivel de enlace	Nivel de transporte
Alcance	Entre dos nodos directamente conectados (enlace físico).	Entre procesos terminales a lo largo de toda la red (extremo a extremo).
Control de flujo	Fijo, entre dos dispositivos, se utilizan MAC	Dinámico, depende del estado del emisor, receptor y la red. Se usan Puertos para identificarse entre procesos
Congestión	No hay congestión (solo un enlace).	Sí hay congestión (la red completa puede saturarse).
Adaptación temporal	Parámetros estáticos o predefinidos. Como el tamaño del paquete y el tipo de detección de errores se determinan en función de la tecnología de enlace	Parámetros adaptativos. Se ajustan el tamaño de la ventana de transmisión y la velocidad de envío de paquetes en tiempo real, dependiendo de las condiciones de la red, como la congestión y la disponibilidad de ancho de banda.
Unidad de datos	Trama (<i>frame</i>).	Segmento.

3 Ejercicio 3

Suponga un protocolo de red que implementa un servicio orientado a conexión usando circuitos virtuales, garantizando la entrega en orden entre dos hosts, pero no realiza control de errores.

¿Podrían dividirse los paquetes de una conexión a nivel de transporte por varios caminos de la red de circuitos virtuales?

El contexto es, tenemos un protocolo que nos obliga a establecer una conexión y en particular, definir un VC , o sea, los paquetes siempre van a viajar a través de él por esa ruta preestablecida. Otro detalle, llegan en orden y no se controlan errores. El tema es que si enviamos los paquetes por caminos separados, corremos el riesgo de que no lleguen en orden debido a las características físicas de esas rutas como la velocidad de transmisión.

Si las conexiones del protocolo de red son simplex (i.e., sólo un host escribe y el otro lee), ¿podrían establecerse conexiones de transporte full-duplex (i.e., los dos leen y escriben)?

En la medida en que exista otro circuito virtual, otra conexión lógica con el sentido inverso, es posible.

¿Sería necesario utilizar números de secuencia en los paquetes de la conexión de transporte?

Sí, ya que dependiendo el simplex, tendremos que distinguir los paquetes en un sentido y otro.

4 Ejercicio 4

Si tuviese que diseñar un sistema de monitoreo de red que deba registrar las conexiones TCP establecidas e identificar al cliente y servidor, utilizando para ello un único paquete dado que posee recursos limitados (no se puede mantener el estado de las conexiones), ¿qué paquete del three-way handshake utilizaría? (Suponiendo que se pueden distinguir todos los paquetes del mismo) ¿Por qué? Justifique su respuesta e indique por qué no es posible utilizar los otros paquetes.

El problema es el siguiente : queremos monitorear las conexiones de la red. Se nos impone una restricción, usar sólo un segmento del three-way-handshake.

1. **SYN** : en este segmento, como en todos tenemos IP src, dst y Puertos src,dst. En particular, tenemos marcado el inicio (por lo menos potencialmente) de una conexión. El tema de usar este segmento es que no tenemos confirmación de la conexión.
2. **SYN/ACK** : contiene la información anterior, y además, tenemos confirmación de que la conexión se pudo establecer.
3. **ACK** : es un segmento interesante porque además de confirmar la conexión, utiliza el número de secuencia del lado del cliente y el acknowledge contiene el $y + 1$. Este paquete, es ideal para confirmar una comunicación y registrarla.

5 Ejercicio 5

Dada la siguiente secuencia de segmentos TCP, responder las preguntas que siguen:

#	ORIG	DEST	FLAGS	#SEQ	#ACK	LENGTH
1	1.2.3.4:5678	20.232.1.1:80	S	0	—	0
2	1.1.1.1:2222	3.3.3.3:4444	S	42	—	0
3	20.232.1.1:80	1.2.3.4:5678	SA	10000	1	0
4	1.2.3.4:5678	20.232.1.1:80	A	1	10001	0
5	3.3.3.3:4444	1.1.1.1:2222	SA	54321	43	0
6	1.2.3.4:5678	20.232.1.1:80	A	1	10001	50
7	1.1.1.1:2222	3.3.3.3:4444	A	43	54322	0
8	20.232.1.1:80	1.2.3.4:5678	A	10001	51	0
9	20.232.1.1:80	1.2.3.4:5678	A	10001	51	200
10	1.1.1.1:2222	3.3.3.3:4444	SAU	43	64334	0
11	20.232.1.1:80	1.2.3.4:5678	F	10201	—	0
12	1.2.3.4:5678	20.232.1.1:80	FA	51	10202	0
13	3.3.3.3:4444	1.1.1.1:2222	RA	54321	43	0
14	20.232.1.1:80	1.2.3.4:5678	A	10202	52	0

1. Asociar cada segmento con cada una de las conexiones indicando el criterio usado para determinar a qué conexión pertenece un segmento. ¿Cuántas son?
2. Detectar el cierre anómalo de una de las conexiones e indicar una posible causa.
3. Para la otra conexión, detallar los cambios de estado en cada extremo a lo largo de toda la comunicación.

Resolución

Para responder cuántas conexiones TCP hay, tal vez sea conveniente mirar primero la segunda pregunta del ejercicio. En general, al momento de determinar la conexión asociada a un paquete dado, hay que mirar cuatro componentes: dirección IP origen, puerto origen, dirección IP destino y puerto destino. Con esta tupla de valores podemos caracterizar el flujo al que pertenece el paquete. En este caso, como los orígenes son distintos, basta con mirar sólo la IP origen. Dicho esto, podemos concluir que en la captura aparecen dos conexiones: (1.2.3.4:5678, 20.232.1.1:80) y (1.1.1.1:2222, 3.3.3.3:4444).

Respecto al cierre anómalo, puede verse que éste ocurre en el paquete 13, cuando el proceso en 3.3.3.3:4444 envía un RST. Esto se debe al paquete con flags SYN/ACK/URG recibido en la línea 10: recibir un SYN en una conexión sincronizada tiene como efecto el envío de un RST, conforme a la especificación de TCP (para más detalles, referirse a la página 71 del RFC 793). La consecuencia de este envío es que el receptor procederá a liberar los recursos de la conexión de inmediato, sin ejercer el algoritmo de cierre tradicional. A su vez, el host emisor del RST hace esto mismo en reacción al paquete mal formado que recibió.

Veamos finalmente cómo responder la última pregunta. Para esto, separemos primero los paquetes de la conexión:

#	ORIG	DEST	FLAGS	#SEQ	#ACK	LENGTH
1	1.2.3.4:5678	20.232.1.1:80	S	0	—	0
3	20.232.1.1:80	1.2.3.4:5678	SA	10000	1	0
4	1.2.3.4:5678	20.232.1.1:80	A	1	10001	0
6	1.2.3.4:5678	20.232.1.1:80	A	1	10001	50
8	20.232.1.1:80	1.2.3.4:5678	A	10001	51	0
9	20.232.1.1:80	1.2.3.4:5678	A	10001	51	200
11	20.232.1.1:80	1.2.3.4:5678	F	10201	—	0
12	1.2.3.4:5678	20.232.1.1:80	FA	51	10202	0
14	20.232.1.1:80	1.2.3.4:5678	A	10202	52	0

Ahora, analicemos paquete por paquete qué está sucediendo en cada extremo:

- Del paquete 1 se desprende que 1.2.3.4:5678 está haciendo una apertura activa. Por ende, este paquete hace que su TCP se mueva de CLOSED a SYN-SENT.
- El paquete 3 es una respuesta positiva del paquete 1 por parte del proceso 20.232.1.1:80. Para que esto ocurra, su TCP debe haber estado en LISTEN antes de poder contestar. Al recibir el SYN, la reacción consiste en trasladarse a SYN-RCVD y emitir un paquete SYN+ACK.
- El primer host recibe luego este segmento y esto hace que se mueva a ESTABLISHED, enviando como respuesta el ACK final del three-way handshake (paquete 4).
- Una vez procesado esto, el TCP de 20.232.1.1:80 se moverá también a ESTABLISHED.

- Los paquetes 6, 8 y 9 corresponden a envío y reconocimiento de datos, por lo que no motivan transiciones entre estados.
- En la porción final de la captura, se inicia el proceso de cierre de conexión. El segmento 11 nos muestra que 20.232.1.1:80 está haciendo un cierre activo, revirtiendo así su rol “pasivo” al momento de establecer la conexión. El efecto de este envío es transicionar al estado **FIN-WAIT-1**.
- En la siguiente línea, su interlocutor recibe este mensaje y reacciona respondiendo un paquete **FIN+ACK** que no sólo reconoce el primer **FIN** (puesto que su **#ACK** es 10202), sino que además inicia el cierre de su stream de escritura. En consecuencia, su TCP colapsa dos estados, pasando de **ESTABLISHED** a **LAST-ACK**.
- El segmento 14 reconoce correctamente el **FIN** de 1.2.3.4:5678. Al recibirlo, el TCP de dicho proceso transicionará a **CLOSED**, mientras que el de 20.232.1.1:80 se moverá a **TIME-WAIT** como respuesta al segmento 12. Asumiendo que el último **ACK** no se extravía en la red, allí permanecerá por 120 segundos (2 MSLs (ver en : MSL - Maximum Segment Lifetime)) antes de pasar también a **CLOSED**.

6 Ejercicio 6

En un Linux, abrir tres consolas:

En una, ejecutar `watch netstat -an`

Esta herramienta que nos permite ver los sockets abiertos del sistema. Las columnas nos indican:

- **Proto:** Protocolo (TCP, UDP)
- **Recv-Q / Send-Q:** Bytes en cola de recepción/envío
- **Local Address:** IP y puerto de tu máquina (origen)
- **Foreign Address:** IP y puerto remoto (destino)
- **State:** Estado TCP: LISTEN, ESTABLISHED, TIME_WAIT, etc.

Corremos entonces con la opción **-an** que muestra todas las conexiones y sockets con direcciones y puertos numéricos.

```
watch netstat -an
```

En otra, utilizar netcat para escuchar en el puerto 5555 de localhost. ¿Qué cambios se observan en la primera?

Lo que vamos a hacer es correr:

```
nc -l 5555
```

Lo que estamos haciendo es crear un socket TCP en nuestra máquina. Y luego, hacer bind a localhost:5555, o sea, ahora el socket ahora “escucha” en el puerto 5555.

Con lo cual el sistema operativo sabe que puede aceptar conexiones entrantes en ese puerto.

Antes no figuraba :

tcp	0	0 0.0.0.0:5555	0.0.0.0:*	LISTEN
-----	---	----------------	-----------	--------

En la restante, utilizar nuevamente netcat para conectarse a dicho puerto en localhost. ¿Qué cambios se observan ahora en la primera?

Vamos a correr :

```
nc localhost 5555
```

Que lo que hace es crear un socket TCP cliente. Luego, intenta conectarse al IP=127.0.0.1, puerto=5555. Como consecuencia, el SO busca un puerto efímero libre para el cliente, por ejemplo 59938. Y finalmente, se completa el three-way handshake TCP entre cliente y listener (SYN → SYN-ACK → ACK).

tcp	0	0 127.0.0.1:5555	127.0.0.1:59938	ESTABLISHED
-----	---	------------------	-----------------	-------------

Cerrar esta última conexión y analizar en qué estado se la marca en la primer consola.

Si cerramos la conexión desde el cliente,

tcp	6	0	127.0.0.1:5555	127.0.0.1:53150	CLOSE_WAIT
-----	---	---	----------------	-----------------	------------

En la Terminal 1, ahora vemos el estado de la conexión como CLOSE_WAIT (lado servidor) y como TIME_WAIT el lado del cliente. Estos estados indican que la conexión está en proceso de cerrarse. TIME_WAIT significa que el cliente ha cerrado la conexión, pero espera un breve período para asegurarse de que todos los paquetes se hayan procesado. CLOSE_WAIT indica que el servidor recibió la solicitud de cierre pero aún no ha finalizado completamente el proceso.

Repetir todo lo anterior usando sockets de Python para ambos procesos.

7 Ejercicio 7

A continuación se presenta una lista de paquetes capturados.

#	Origen	Destino	Flags	Seq	ACK	Length
01	3.14.15.92:654	2.71.82.81:823	S	1	—	0
02	2.71.82.81:823	3.14.15.92:654	SA	1	2	0
03	3.14.15.92:654	2.71.82.81:823	A	2	2	0
04	3.14.15.92:654	2.71.82.81:823		2	—	300
05	2.71.82.81:823	3.14.15.92:654	A	2	302	0
06	3.14.15.92:654	2.71.82.81:823		302	—	1000
07	3.14.15.92:654	2.71.82.81:823		302	—	1000
08	2.71.82.81:823	3.14.15.92:654	A	2	1302	0
09	2.71.82.81:823	3.14.15.92:654	F	2	—	0
10	3.14.15.92:654	2.71.82.81:823	A	1302	3	0
11	3.14.15.92:654	2.71.82.81:823	F	1302	—	0
12	2.71.82.81:823	3.14.15.92:654	A	3	1303	0

Detalle las transiciones de estados de la conexión TCP para cada uno de los hosts, a partir del segmento 9 en adelante

- **09:** El Host 2, en estado ESTABLISHED, solicita el cierre enviando un segmento con el flag FIN. Pasa a estado FIN_WAIT_1.
- **10:** El host 1, en estado ESTABLISHED, recibe el segmento con el flag FIN y responde con el flag ACK. Transiciona a CLOSE_WAIT.
- **11:** El host 1 envía un segmento con FIN y pasa a LAST_ACK.
- **12:** Host 2 recibe ACK del host 1 y pasa a FIN_WAIT_2. Luego recibe FIN, envía ACK, y pasa a TIME_WAIT, para un tiempo despues pasar a CLOSED. El Host 1 espera el ACK del host 2 para pasar a CLOSED.

Indique qué segmentos se pueden usar para la estimación del RTT.

El RTT (Round-Trip Time) se mide como el tiempo entre:

- El envío de un segmento con datos (o SYN)
- La recepción del ACK correspondiente.

Entonces, se pueden usar los pares donde un paquete con datos es reconocido con un ACK posterior (sin retransmisiones ni cierres).

Es decir :

- **Handshake TCP** : $1 \rightarrow 2 (SYN \rightarrow SYN - ACK)$
- **Datos normales** : $4 \rightarrow 5 (Datos\ 300\ bytes \rightarrow ACK)$

8 Ejercicio 8

Utilizar nmap para realizar un SYN scan al gateway de la red local. Mientras tanto, abrir Wireshark y observar cómo los paquetes son enviados.

Primero vamos a descargar nmap con

```
sudo snap install nmap
```

Algunos de los comandos útiles de nmap son :

- **-sS** : SYN scan (half-open).
- **-p 1-1024** : escanear puertos 1–1024.
- **-Pn** : no hacer host discovery (evita que nmap descarte si ICMP está bloqueado).
- **-v** : verbose, se ve el progreso en la consola.

La forma de uso es

```
Usage:  nmap [Scan Type(s)] [Options] target specification
```

La idea del ejercicio entonces es:

1. **Hacer un SYN scan al gateway**: enviar paquetes SYN a varios puertos del gateway para ver qué responde (sin completar la conexión).
2. **Capturar paquetes**: grabar todo el tráfico con Wireshark mientras hacemos el scan.
3. Observar cómo se comporta TCP:
 - $SYN \rightarrow SYN - ACK$: puerto abierto (host responde).
 - $SYN \rightarrow RST$: puerto cerrado.
 - SYN sin respuesta : filtrado (firewall / drop).
 - Retransmisiones : el escáner reenvía SYN porque no recibe respuesta; indica pérdida o filtrado.
4. Responder las preguntas.

Scan

Lo primero que vamos a hacer es averiguar la ip del gateway

```
ip route show default
```

Luego, nos vamos a wireshark, que lo tenemos que abrir con privilegios elevados, así que

```
sudo wireshark
```

Seleccionamos la **interfaz correcta** (fijarse que luego de correr lo anterior se van a dar cuenta cuál es).

Ahora bien, si corremos

```
sudo nmap -sS (ip gateway)
```

En mi caso falla porque mi wifi no deja que dnsmasq abra la interfaz. Por tal motivo :

```
sudo nmap -sT -Pn -n (ip gateway)
```

Donde :

- **-sT** : TCP connect scan (no raw SYNs).
- **-Pn** : no host discovery (no ARP/ping; evita que nmap intente abrir la interfaz con dnsmasq).
- **-n** : no resolución DNS (más rápido).

¿Usan el mismo puerto origen?

Cuando hacemos el scan se lanzan miles de segmentos TCP y en general usamos muchos puertos diferentes (nmap utiliza puertos aleatorios en el rango 1024-65535).

TCP	74	46612	→	135	[SYN]	Seq=0	Win=64240	Len=0	MSS=1460	SACK_PERM	TSval=896788605	1
TCP	74	37404	→	587	[SYN]	Seq=0	Win=64240	Len=0	MSS=1460	SACK_PERM	TSval=896788605	1
TCP	74	36292	→	22	[SYN]	Seq=0	Win=64240	Len=0	MSS=1460	SACK_PERM	TSval=896788605	TS
TCP	74	50048	→	445	[SYN]	Seq=0	Win=64240	Len=0	MSS=1460	SACK_PERM	TSval=896788605	1
TCP	74	50230	→	80	[SYN]	Seq=0	Win=64240	Len=0	MSS=1460	SACK_PERM	TSval=896788605	TS
TCP	74	56440	→	443	[SYN]	Seq=0	Win=64240	Len=0	MSS=1460	SACK_PERM	TSval=896788605	1
TCP	74	53376	→	554	[SYN]	Seq=0	Win=64240	Len=0	MSS=1460	SACK_PERM	TSval=896788605	1
TCP	74	60790	→	53	[SYN]	Seq=0	Win=64240	Len=0	MSS=1460	SACK_PERM	TSval=896788605	TS
TCP	74	58426	→	8888	[SYN]	Seq=0	Win=64240	Len=0	MSS=1460	SACK_PERM	TSval=896788605	TS
TCP	74	35338	→	993	[SYN]	Seq=0	Win=64240	Len=0	MSS=1460	SACK_PERM	TSval=896788606	1
TCP	66	80	→	50230	[SYN, ACK]	Seq=0	Ack=1	Win=29200	Len=0	MSS=1460	SACK_PERM	WS=64
TCP	54	50230	→	80	[ACK]	Seq=1	Ack=1	Win=64256	Len=0			
TCP	66	443	→	56440	[SYN, ACK]	Seq=0	Ack=1	Win=29200	Len=0	MSS=1460	SACK_PERM	WS=64
TCP	54	56440	→	443	[ACK]	Seq=1	Ack=1	Win=64256	Len=0			
TCP	66	53	→	60790	[SYN, ACK]	Seq=0	Ack=1	Win=29200	Len=0	MSS=1460	SACK_PERM	WS=64
TCP	54	60790	→	53	[ACK]	Seq=1	Ack=1	Win=64256	Len=0			
TCP	60	8888	→	58426	[RST, ACK]	Seq=1	Ack=1	Win=0	Len=0			
TCP	54	50230	→	80	[RST, ACK]	Seq=1	Ack=1	Win=64256	Len=0			
TCP	54	56440	→	443	[RST, ACK]	Seq=1	Ack=1	Win=64256	Len=0			
TCP	54	60790	→	53	[RST, ACK]	Seq=1	Ack=1	Win=64256	Len=0			

¿Qué ocurre con el tamaño de la ventana?

El tamaño de la ventana TCP (campo Window size en el encabezado del paquete TCP) en un SYN-ACK que el gateway envía como respuesta a un paquete SYN:

```
Source Port: 80
Destination Port: 50230
[Stream index: 7]
  ▶ [Conversation completeness: Complete, NO_DATA (39)]
  [TCP Segment Len: 0]
  Sequence Number: 0      (relative sequence number)
  Sequence Number (raw): 1698848697
  [Next Sequence Number: 1      (relative sequence number)]
  Acknowledgment Number: 1      (relative ack number)
  Acknowledgment number (raw): 244488271
  1000 .... = Header Length: 32 bytes (8)
  ▶ Flags: 0x012 (SYN, ACK)
  Window: 29200
  [Calculated window size: 29200]
  Checksum: 0x4b2b [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  ▶ Options: (12 bytes), Maximum segment size, No-Operation (NOP), No-Operation (
  ▶ [Timestamps]
  ▶ [SEQ/ACK analysis]
```

Por supuesto, que en el contexto del SYN-scan el segmento que enviamos nosotros con el SYN no tiene mucho sentido porque la conexión no se establece.

¿Y con el número de secuencia inicial?

Es medio lo mismo que antes, el nro de secuencia, en un SYN es algo totalmente aleatorio que no tiene función si la conexión no se establece.

9 Ejercicio 9

El siguiente fragmento muestra parte de la salida de una ejecución de netstat en un host con dirección IP 192.168.1.104:

Proto	Local Address	Foreign Address	State
tcp	0.0.0.0:7777	0.0.0.0:*	LISTEN
tcp	192.168.1.104:48600	200.42.33.121:80	ESTABLISHED
tcp	192.168.1.104:34688	69.163.203.254:80	TIME-WAIT
tcp	192.168.1.104:45911	173.194.42.245:443	ESTABLISHED

En el transcurso de los cinco minutos siguientes se capturó el tráfico TCP entrante y saliente de dicho host. La siguiente traza corresponde a la porción final de este tráfico.

No.	Source	Destination	Info
1	192.168.1.247	192.168.1.104	27351 > 7777 [SYN] Seq=0 Len=0
2	192.168.1.104	200.42.33.121	GET /index.html HTTP/1.1
3	173.194.42.245	192.168.1.104	443 > 45911 [ACK] Seq=4562 Ack=2347 Len=0
4	192.168.1.104	192.168.1.247	7777 > 27351 [SYN, ACK] Seq=0 Ack=1 Len=0
5	192.168.1.247	192.168.1.104	27351 > 7777 [ACK] Seq=1 Ack=1 Len=0
6	200.42.33.121	192.168.1.104	80 > 48600 [ACK] Seq=232 Ack=104 Len=0
7	200.42.33.121	192.168.1.104	HTTP/1.1 200 OK
8	173.194.42.245	192.168.1.104	443 > 45911 [FIN, ACK] Seq=4562 Ack=2347 Len=0
9	192.168.1.104	173.194.42.245	45911 > 443 [ACK] Seq=2347 Ack=4563 Len=0

- **Estado de los sockets en netstat:**

- **Escucha (LISTEN):** El socket en el puerto 7777 está en estado LISTEN, indicando que espera una conexión entrante. La dirección local 0.0.0.0 significa que escucha en todas las interfaces del host, aceptando conexiones desde cualquier IP.
- **Conexiones establecidas (ESTABLISHED):** Las conexiones TCP con los hosts 200.42.33.121 (puerto 80) y 173.194.42.245 (puerto 443) están activas y han intercambiado datos.
- **TIME-WAIT:** La conexión con 69.163.203.254 en el puerto 34688 probablemente finalizó su intercambio de datos y espera un tiempo determinado para asegurarse de que no queden paquetes huérfanos.

- **Paquetes durante los 5 minutos:**

- **Paquete 1:** SYN desde 192.168.1.247 a 192.168.1.104. El host 192.168.1.247 inicia la conexión TCP al puerto 7777. Seq=0, Len=0.
- **Paquete 2:** Solicitud HTTP de 192.168.1.104 a 200.42.33.121. El host pide la página /index.html en el puerto 80.
- **Paquete 3:** ACK desde 173.194.42.245 a 192.168.1.104. Confirma que la conexión HTTPS (puerto 443) está establecida.
- **Paquete 4:** SYN, ACK de 192.168.1.104 a 192.168.1.247. Respuesta al SYN inicial del paquete 1, indicando disposición para establecer conexión en el puerto 7777 y confirmando recepción del SYN.
- **Paquete 5:** ACK de 192.168.1.247 a 192.168.1.104. Último paso del handshake TCP, conexión completamente establecida.
- **Paquete 6:** ACK de 200.42.33.121 a 192.168.1.104. Parte de la conexión HTTP establecida (puerto 80), confirmando recepción de datos.
- **Paquete 7:** Respuesta HTTP de 200.42.33.121 a 192.168.1.104. Contiene la respuesta 200 OK al GET solicitado en el paquete 2.
- **Paquete 8:** FIN, ACK de 173.194.42.245 a 192.168.1.104. Indica que el host cierra la conexión TCP (puerto 443).
- **Paquete 9:** ACK de 192.168.1.104 a 173.194.42.245. Confirma recepción del FIN, ACK y completa el cierre ordenado de la conexión.

Indicar qué se obtendría si se volviese a correr netstat una vez transcurrido este tiempo

- Las conexiones que estaban en **TIME-WAIT** probablemente ya no aparecerán, ya que este estado dura un tiempo fijo (generalmente $2 \times \text{MSL}$, entre 1 y 4 minutos).
- Los sockets en **ESTABLISHED** seguirán visibles si la conexión sigue activa.
- Los sockets en **LISTEN** permanecen sin cambios, esperando nuevas conexiones entrantes.

A partir del paquete 1, el proceso corriendo en 192.168.1.247:27351 está iniciando una conexión con nuestro host. ¿Cuál es el instante más temprano posible en el que el proceso receptor puede enviar datos a su contraparte?

- Paquete 1: SYN desde 192.168.1.247 hacia nuestro host (puerto 7777), iniciando la conexión.
- La conexión TCP se establece tras completar el **handshake**
- **Conclusión:** El proceso receptor puede enviar datos inmediatamente después de recibir el ACK final del cliente, es decir, al finalizar el handshake (paquete 5), ya que antes la conexión no está establecida.

10 Ejercicio 10

Desde un host con dirección 192.168.0.1 se captura el siguiente tráfico:

No.	Time	Source	Destination	Info
1	5.487931	192.168.0.100	157.92.27.21	60205 > http [SYN] Seq=0 Len=0
2	5.505790	157.92.27.21	192.168.0.100	http > 60205 [SYN, ACK] Seq=0 Ack=1 Len=0
3	5.505848	192.168.0.100	157.92.27.21	60205 > http [ACK] Seq=1 Ack=1 Len=0
4	5.506098	192.168.0.100	157.92.27.21	GET /materias/tc/2012/2c/index_html HTTP/1.1
5	5.532603	157.92.27.21	192.168.0.100	http > 60205 [ACK] Seq=1 Ack=568 Len=0
6	5.930460	157.92.27.21	192.168.0.100	[TCP segment of a reassembled PDU]
7	5.930494	192.168.0.100	157.92.27.21	60205 > http [ACK] Seq=568 Ack=1449 Len=0
8	5.930573	157.92.27.21	192.168.0.100	[TCP segment of a reassembled PDU]
9	5.930576	192.168.0.100	157.92.27.21	60205 > http [ACK] Seq=568 Ack=2897 Len=0

Describir los cambios de estado de ambos extremos de la conexión

- **Paquete 1 (SYN 60205 → http):**
 - 192.168.0.100: CLOSED → SYN-SENT
 - 157.92.27.21: LISTEN → SYN-RECEIVED
- **Paquete 2 (SYN, ACK http → 60205):**
 - 192.168.0.100: recibe SYN-ACK, se prepara para ESTABLISHED
 - 157.92.27.21: sigue en SYN-RECEIVED hasta recibir ACK
- **Paquete 3 (ACK 60205 → http):** handshake completo, ambos pasan a ESTABLISHED
- **Paquete 4 (GET HTTP):** 192.168.0.100 envía datos, ambos en ESTABLISHED
- **Paquetes 5-8 (ACKs):** intercambio de ACKs, conexión permanece ESTABLISHED

Explicar en qué estado quedan ambos extremos de la conexión si un software en el host 192.168.0.1 (gateway de 192.168.0.100) reemplaza maliciosamente el segmento 9 por el siguiente:

192.168.0.100 157.92.27.21 60205 > http [RST, ACK] Seq=568 Ack=2897 Len=0

- Segmento 9: 60205 → http [RST, ACK]
- 157.92.27.21: recibe RST, socket pasa a CLOSED, descarta datos pendientes
- 192.168.0.100: al enviar RST, también pasa a CLOSED
- **Resultado:** ambos extremos quedan en **CLOSED**, conexión terminada abruptamente

11 Ejercicio 11

Consideremos un host con dirección IP 10.80.1.10. En el instante t_0 , una ejecución de netstat en este host arroja lo siguiente:

Proto	Local Address	Foreign Address	State
tcp	0.0.0.0:8080	0.0.0.0:*	LISTEN
tcp	10.80.1.10:35336	157.92.27.21:80	SYN-SENT
tcp	10.80.1.10:51247	200.42.93.137:80	TIME-WAIT
tcp	10.80.1.10:24592	98.139.183.24:80	ESTABLISHED

Al cabo de un momento, en el instante t_1 se vuelve a correr netstat y esta vez la salida es así:

Proto	Local Address	Foreign Address	State
tcp	0.0.0.0:8080	0.0.0.0:*	LISTEN
tcp	10.80.1.10:35336	157.92.27.21:80	CLOSE-WAIT
tcp	10.80.1.10:8080	200.11.54.101:32154	ESTABLISHED
tcp	10.80.1.10:24592	98.139.183.24:80	ESTABLISHED

Proponer un intercambio de paquetes TCP entre el host dado y los demás de manera tal que, partiendo de t_0 , la ejecución de netstat en t_1 sea tal como se describe arriba.

- **Socket 10.80.1.10:35336 $\rightarrow$ 157.92.27.21:80:**
 - t_0 : cliente envía SYN
 - $t_0 + \Delta t$: servidor responde FIN
 - Host local pasa a **CLOSE-WAIT**, esperando cerrar su extremo
- **Socket 10.80.1.10:51247 $\rightarrow$ 200.42.93.137:80:**
 - Estado TIME-WAIT en t_0
 - Después de $2 \times \text{MSL}$, socket desaparece de netstat
- **Nuevo socket 10.80.1.10:8080 $\rightarrow$ 200.11.54.101:32154:**
 - Se realiza handshake TCP completo: SYN $\rightarrow$ SYN-ACK $\rightarrow$ ACK
 - Estado final: ESTABLISHED
- **Socket 10.80.1.10:24592 $\rightarrow$ 98.139.183.24:80:** permanece ESTABLISHED

Si se sabe que el último paquete enviado al host 200.42.93.137 se dio en t_0 , indicar una cota inferior para t_1 expresada en términos de t_0 .

- Dado que el socket hacia 200.42.93.137 estaba en TIME-WAIT en t_0 ,
- El tiempo mínimo que debe transcurrir para que desaparezca es $2 \times \text{MSL}$
- Por lo tanto, $t_1 \geq t_0 + 2 \times \text{MSL}$ (para refrescar : MSL - Maximum Segment Lifetime)

12 Ejercicio 12

La técnica de port knocking permite abrir puertos dinámicamente en un sistema operativo a partir de sucesivos intentos de conexión a una secuencia de puertos TCP preestablecidos. Supongamos que un host con dirección IP 192.168.0.100 implementa esta técnica y la secuencia de puertos elegida para abrir el puerto 2000 es 5000, 10500, 20000. En otras palabras, esto significa que cualquier host que desee establecer una conexión al puerto 2000 de 192.168.0.100 debe primero generar una secuencia de paquetes a esos puertos, que asumimos cerrados, y en ese exacto orden. Una vez detectado esto, el sistema operativo del host procederá a abrir el puerto 2000.

Supongamos que un host con dirección IP 192.168.10.10 desea conectarse al puerto 2000 de 192.168.0.100, pero todos los paquetes que envía luego del port knocking se pierden en la red. Indicar qué eventos ocurren desde el punto de vista de 192.168.10.10 desde el momento de iniciar el proceso de conexión.

- 192.168.10.10 envía SYN al puerto 5000 (primer knock). Recibe RST o no respuesta.
- Envía SYN al puerto 10500 (segundo knock). Recibe RST o no respuesta.
- Envía SYN al puerto 20000 (tercer knock). Recibe RST o no respuesta. El servidor detecta la secuencia y abre el puerto 2000.
- 192.168.10.10 envía SYN al puerto 2000 pero ese SYN (o la respuesta) se pierde en la red.
- El socket cliente queda en **SYN-SENT** y ejecuta retransmisiones TCP (reenvío del SYN con backoff exponencial).
- Tras agotar retransmisiones/timeouts el stack TCP falla la conexión y el socket local vuelve a **CLOSED** (error de timeout en la API).
- **Resumen:** se observan SYN retransmitidos y finalmente timeout; aunque el puerto 2000 quedó abierto en el servidor, la conexión no se llega a establecer por pérdidas en la red.

Supongamos ahora que el port knocking tuvo éxito y el puerto 2000 está abierto. Proponer un intercambio de a lo sumo ocho paquetes TCP entre ambos hosts que permita que 192.168.10.10 envíe el mensaje 'HOLA' y 192.168.0.100 le responda 'HOLA'. Además, ambos hosts deben atravesar en algún momento el estado CLOSING.

1. $C \rightarrow S : [SYN]$.
2. $S \rightarrow C : [SYN, ACK]$.
3. $C \rightarrow S : [ACK]$. (Handshake completo; ambos en ESTABLISHED)
4. $C \rightarrow S : [PSH, ACK]$ con payload "HOLA". (cliente envía "HOLA")
5. $S \rightarrow C : [ACK]$. (ack del servidor al "HOLA" recibido)
6. $S \rightarrow C : [PSH, ACK]$ con payload "HOLA". (servidor responde "HOLA")
7. $C \rightarrow S : [FIN, ACK]$. (cliente inicia cierre)
8. $S \rightarrow C : [FIN, ACK]$. (servidor también inicia cierre casi simultáneamente)

Si los FINs se cruzan (cada extremo envía FIN antes de recibir el ACK del FIN propio), ambos extremos entran en el estado **CLOSING** de forma transitoria. Posteriormente se intercambian los ACKs finales y la conexión se completa su cierre (uno de los lados acaba en TIME-WAIT según quién sea el que terminó realizando el cierre activo).

13 MSL - Maximum Segment Lifetime

Es el tiempo máximo que un segmento TCP puede permanecer "vivo" en la red antes de ser descartado. TCP utiliza MSL para asegurarse de que paquetes antiguos no interfieran con nuevas conexiones que usen los mismos puertos y números de secuencia.

TIME-WAIT y $2 \times \text{MSL}$

Cuando una conexión TCP se cierra de forma ordenada, el lado que cierra primero entra en el estado **TIME-WAIT**. El socket se mantiene en TIME-WAIT durante **dos veces el MSL** ($2 \times \text{MSL}$). Esto garantiza que:

- Todos los ACK finales tengan tiempo de llegar.
- Paquetes antiguos que se retrasen en la red no interfieran con una nueva conexión que use los mismos puertos y números de secuencia.

Ejemplo: si $\text{MSL} = 30 \text{ s}$, entonces $\text{TIME-WAIT} = 2 \times 30 = 60 \text{ s}$. Durante esos 60 s, **netstat** aún muestra el socket en TIME-WAIT.

Conclusión: $2 \times \text{MSL}$ representa el tiempo que TCP espera antes de eliminar un socket cerrado, previniendo colisiones de paquetes antiguos.